

KMS: key types, envelope encryption, and the three ways to grant access

Domain 5 opens here. The key policy is the root of trust, and grants are the part your question bank cares about most.

The key policy is the **root of trust**. Without it, no IAM policy anywhere grants access to that key.

Three key types

TYPE	WHO CONTROLS IT	VISIBLE TO YOU?
AWS owned	AWS. Shared across many accounts	No. Not in your account, no CloudTrail for you
AWS managed	AWS, on your behalf. Named aws/service	Yes, in your account. Policy is AWS's. Free
Customer managed	You. Full control of policy and rotation	Yes. Charged monthly

ON THE EXAM

'We must control the key' or 'we must be able to revoke access' means customer managed. Nothing else gives you the policy.

Envelope encryption

01

Request

Your application calls `GenerateDataKey` against a KMS key.

02

Two copies

KMS returns the data key in plaintext and encrypted.

03

Encrypt

You encrypt the data locally with the plaintext data key.

04

Discard

You wipe the plaintext key and store the encrypted one beside the data.

05

Decrypt

Later, send the encrypted data key back to KMS to unwrap it.

ON THE EXAM

KMS never sees your data and never stores the data key. Direct Encrypt is capped at 4 KB, which is why envelope encryption exists.

The key policy, and the IAM delegation statement

- ◆ **Every KMS key has a key policy.** Create one without specifying it and AWS attaches a default.
- ◆ The default holds an **Enable IAM User Permissions** statement – Principal the account root, Action kms:.*.
- ◆ **That statement delegates to IAM.** It grants nobody anything; it permits IAM policies to grant.
- ◆ **Remove it and IAM policies stop working for that key,** whatever they say.
- ◆ It is also how you lock yourself out. Only AWS Support can recover a key.

ON THE EXAM

One statement explains why most keys behave normally and a few do not.

Three ways to allow access

- ◆ **Key policy** – names principals directly. Always required, and sufficient on its own.
- ◆ **IAM policy** – works only if the key policy delegates to IAM. Convenient for many principals across many keys.
- ◆ **Grants** – temporary, programmatic, granular. Created by API, not by editing a policy.
- ◆ Grants exist because AWS services need to use your key on your behalf, briefly, without anyone editing a policy document.
- ◆ **kms:CreateGrant permission comes from the key policy.** Whoever holds it can extend access to principals you did not choose.

ON THE EXAM

Key policy and IAM are declarative and durable. Grants are programmatic and temporary. The exam tests when each is right.

Grants: what they actually are

- ◆ Gives a **grantee principal** permission for a specific list of **operations**, optionally constrained.
- ◆ Principals: accounts, IAM users, roles, federated and assumed-role users. **Cross-account works.**
- ◆ Operations must suit the key type – no Sign or GenerateMac on a symmetric encryption key.
- ◆ Deleting a grant removes every permission it gave, in one call.
- ◆ **50,000 grants per key.** High, and still a ceiling.

ON THE EXAM

A grant is created and destroyed by code, at the pace the workload runs.

Grant tokens and eventual consistency

- ◆ **Creating, retiring or revoking a grant is eventually consistent** – usually under five minutes before it applies everywhere.
- ◆ **CreateGrant returns a GrantToken**, and passing that token gives you the permission **immediately**, before consistency is reached.
- ◆ **CreateGrant is the only operation that returns a grant token.**
- ◆ Once the grant is consistent, KMS uses the grant itself and the token is no longer needed.
- ◆ So a workflow that creates a grant and uses the key in the next second **must** pass the token, or it will intermittently fail.

ON THE EXAM

'Works when we retry, fails when we do not' is the signature. That is eventual consistency, and the token is the fix.

Grant constraints and encryption context

- ◆ **EncryptionContextEquals** – the request's encryption context must match exactly.
- ◆ **EncryptionContextSubset** – the request's context must **include** the pairs you specify, and may have others.
- ◆ So a grant can permit decryption of one tenant's data only, by constraining on a tenant identifier in the context.
- ◆ Up to 8 encryption context pairs per constraint.
- ◆ **Constraints appear in plaintext in CloudTrail.** AWS says explicitly: do not put confidential information in them.

ON THE EXAM

Encryption context is additional authenticated data. It is not secret, it is not optional once used, and it is logged.

Retiring and revoking

- ◆ Both delete the grant. The difference is who does it, and why.
- ◆ **Retire** – the grant is finished with. The **retiring principal** nominated at creation may do it.
- ◆ **Revoke** – an administrator removes a grant that should not exist. Needs kms:RevokeGrant.
- ◆ **ListGrants** and **ListRetirableGrants** find them. Both operations need the GrantId.
- ◆ **Both are eventually consistent**, so a revoked grant may work briefly afterwards.

ON THE EXAM

Revoking a grant is not instantaneous. Neither is editing a key policy.

Condition keys worth knowing

- ◆ **kms:ViaService** – the request must arrive **through a named AWS service**, not directly.
- ◆ That is how you allow reading encrypted S3 objects without allowing kms:Decrypt on arbitrary ciphertext.
- ◆ **kms:CallerAccount** – the caller's account. **kms:EncryptionContext:** – match a context pair.
- ◆ **kms:KeySpec, kms:KeyOrigin, kms:KeyUsage** – constrain what kinds of key may be created or used.
- ◆ Module 1: **aws:SourceArn is not supported** on CloudTrail Lake event data store key policies.

ON THE EXAM

ViaService separates 'may use the key through this service' from 'may use the key'.

A data science role in a single account has an IAM policy allowing kms:Decrypt on Resource *, and the AWS managed AdministratorAccess policy. It calls kms:Decrypt against a customer managed key in the same account and receives AccessDenied.

The key is enabled, in the same Region, and not scheduled for deletion. The key policy contains statements naming two other roles, and nothing else.

Why is the call denied?

- ☐ A The key policy does not delegate to IAM, so no IAM policy grants access to this key
- ☐ B kms:Decrypt cannot be granted on Resource *; it requires a specific key ARN
- ☐ C AdministratorAccess explicitly excludes KMS actions
- ☐ D The role needs a grant, because IAM policies never apply to customer managed keys

◆ PAUSE AND DECIDE

Why is the call denied?

- A

The key policy does not delegate to IAM

Correct. The key policy names two other roles and contains no Enable IAM User Permissions statement, so IAM policies confer nothing on this key.
- B

kms:Decrypt requires a specific key ARN

*Resource * is valid and common in KMS policies. The scope of the IAM policy is not the problem – its relevance is.*
- C

AdministratorAccess excludes KMS

False. It is Allow on all actions and all resources. KMS access can still fail, and not because of that policy's contents.
- D

IAM policies never apply to customer managed keys

Too strong, and it is the misconception the correct answer refines. IAM policies work perfectly well – when the key policy delegates to them.

THE SIGNAL

'The key policy names two other roles, and nothing else' is the whole question. No delegation statement, no IAM.

A multi-tenant SaaS platform encrypts every tenant's data under one customer managed key, with an encryption context carrying tenant identifiers. A batch job must decrypt one tenant's data for a migration lasting about six hours.

Security requires that the job can decrypt only that tenant's data, that access ends when the job does, and that no key policy or IAM policy is edited to enable it.

What should the platform use?

- ☐ A Add a statement to the key policy allowing the job role kms:Decrypt, and remove it afterwards
- ☐ B Attach an IAM policy to the job role with a kms:EncryptionContext condition
- ☐ C A grant to the job role for Decrypt, with an EncryptionContextSubset constraint on the tenant, retired when the job finishes
- ☐ D A session policy passed when the job assumes its role, restricting kms:Decrypt to that tenant

◆ PAUSE AND DECIDE

What should the platform use?

- A

 Edit the key policy, then revert
Explicitly ruled out by the requirement, and it is what grants exist to avoid. Editing a shared key policy for one job is exactly the risk.
- B

 IAM policy with an encryption context condition
Would work and is durable, not temporary. It also edits an IAM policy, which the requirement forbids, and needs removing afterwards.
- C

 A grant with EncryptionContextSubset, retired when done
Correct. Programmatic, scoped to the tenant by constraint, and removable in one call – which is precisely why grants exist.
- D

 A session policy restricting kms:Decrypt
A session policy can only narrow permissions the role already has. If the role cannot decrypt, a session policy cannot give it that.

THE SIGNAL

Three requirements – scoped, temporary, no policy edits – are the three properties that define a grant.

A media pipeline creates a KMS grant for a transcoding role and immediately calls `GenerateDataKey` using that role. Roughly one job in five fails with `AccessDenied`. Retrying the same job a few minutes later always succeeds.

The grant is created correctly each time, with the right operations and grantee. Nothing in the key policy, the IAM policies or the SCPs changes between the failing and succeeding attempts.

What is happening, and what is the fix?

- ☐ A The grant limit of 50,000 per key is being reached; delete old grants
- ☐ B `GenerateDataKey` is not a valid grant operation; use `Encrypt` instead
- ☐ C The transcoding role needs `kms:CreateGrant` in its IAM policy as well
- ☐ D Grants are eventually consistent; pass the `GrantToken` returned by `CreateGrant`

◆ PAUSE AND DECIDE

What is happening, and what is the fix?

- A

Hitting the 50,000 grant limit

That produces a consistent failure at a threshold, not an intermittent one – and it would not resolve itself on a retry minutes later.
- B

GenerateDataKey is not a valid grant operation

It is valid for a symmetric encryption key. Sign and GenerateMac would not be – but then it would fail every time, not one in five.
- C

The role needs kms:CreateGrant

CreateGrant is what the pipeline already did successfully. The role is the grantee here; it does not need permission to create grants.
- D

Eventual consistency; pass the GrantToken

Correct. Grant creation takes up to about five minutes to propagate. The token from CreateGrant gives a strongly consistent check immediately.

THE SIGNAL

Intermittent failure that a retry fixes is eventual consistency. A permissions problem fails every time.

Analysts must read encrypted objects from a reporting bucket, which requires decryption under a customer managed key. They should not be able to call kms:Decrypt directly against arbitrary ciphertext, because the same key protects other datasets.

The security team wants one control that permits the S3 path and blocks the direct one, without maintaining a list of object ARNs.

What should the team add to the key policy?

- A A Deny on kms:Decrypt for the analyst role, with an exception listing the reporting bucket's objects
- B An Allow for the analyst role conditioned on kms:ViaService equal to the S3 endpoint in that Region
- C A grant to the analyst role with an EncryptionContextEquals constraint on the bucket name
- D An Allow conditioned on aws:SourceVpce matching the S3 gateway endpoint

◆ PAUSE AND DECIDE

What should the team add to the key policy?

- A

 Deny with an object exception list
KMS policies do not evaluate S3 object ARNs, and the requirement rules out maintaining a list anyway.
- B

 Allow conditioned on kms:ViaService for S3
Correct. ViaService permits the key's use only when the request arrives through S3 acting for the principal, so a direct kms:Decrypt call is not permitted.
- C

 Grant with EncryptionContextEquals on the bucket
The closest wrong answer – S3 does use the object ARN as encryption context. It scopes which objects, not which path, so direct Decrypt with the right context still works.
- D

 Allow conditioned on aws:SourceVpce
Constrains the network path, not the calling service. An analyst inside that VPC could still call kms:Decrypt directly.

THE SIGNAL

'Through this service' and 'from this network' are different questions. ViaService answers the first; SourceVpce answers the second.

Six things to carry forward

- ◆ **The key policy is the root of trust.** No delegation statement means no IAM policy works.
- ◆ **Policies are durable; grants are temporary and programmatic.**
- ◆ **Grants are eventually consistent** – the GrantToken is the immediate path.
- ◆ **EncryptionContextSubset scopes a grant to a tenant.** Constraints are logged in plaintext.
- ◆ **kms:ViaService separates 'through this service' from 'directly'.**
- ◆ **Revoking a grant is eventually consistent too.** Not an instant containment action.

ON THE EXAM

Next: rotation, multi-Region keys, imported key material, external key stores and CloudHSM.